7. Generative AI, Part 1

How large language models work

Week 7 · CE/ARCE 2XX AI Literacy for Civil and Architectural Engineering



COLLEGE OF ENGINEERING
Civil & Architectural Engineering & Mechanics

Agenda

Today's plan

1. Quiz on Week 6 (warm-up for the midterm)
2. From predicting to generating: the next-token idea
3. Inside the model: tokens, embeddings, attention
4. How an LLM is trained — and why scale matters
5. Generating text — and why LLMs hallucinate
6. Lab day: midterm exam logistics

Recap

Last week in 60 seconds

1 Convolution

A small filter slides over an image and lights up at edges and patterns — the feature map.

2 CNN stack

Conv → pool → … → dense. Early layers find edges; later layers find parts like cracks or spalls.

3 Transfer learning

Reuse a network pretrained on millions of images; train only a small head (Lab 06's crack detector).

4 Sequences

RNNs and LSTMs carry a memory along a time series. Attention lets every element look at every other — today.

Learning objectives

By the end of today, you can...

- ✓ Explain language modeling as **next-token prediction**, and how repeating it generates text
- ✓ Describe **tokens** and **embeddings**, and why "superplasticizer" becomes four tokens
- ✓ Explain in plain words what **attention** does and what a **transformer block** contains
- ✓ Describe the training stages — **pretraining, instruction tuning, preference tuning** — and the role of scale
- ✓ Explain how **temperature** and the **context window** shape output, and why LLMs **hallucinate**

01

Predicting the next token

The one idea behind every large language model

Next-token prediction

Generative models create new content instead of one label

Predictive models (Weeks 3–6)

- Input: a concrete mix → output: **strength in MPa**
- Input: a photo → output: **crack / no crack**
- One answer per input; easy to score right or wrong

vs

Generative models (Weeks 7–8)

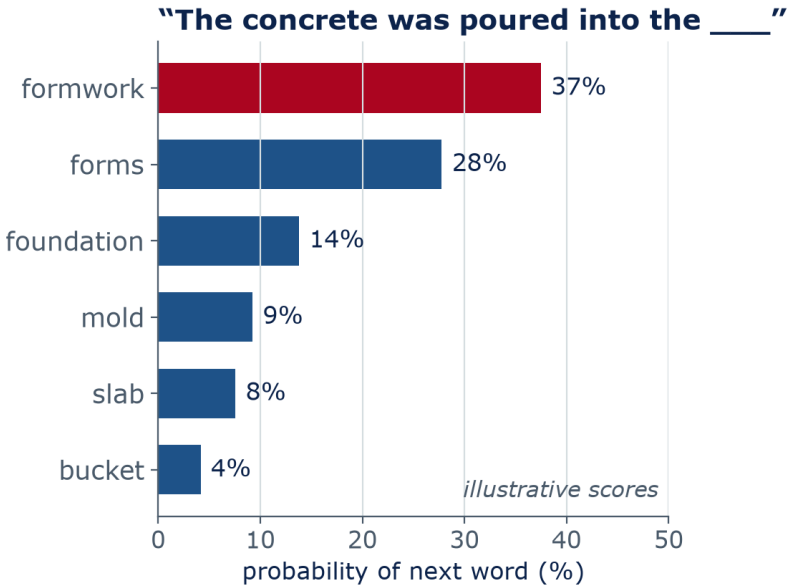
- Input: a prompt → output: **new text, code, or images**
- Built piece by piece; many plausible answers
- Hard to score: “fluent” is not the same as “correct”

Surprise: underneath, an LLM is a classifier — at each step it picks the next token from ~150,000 classes

Next-token prediction

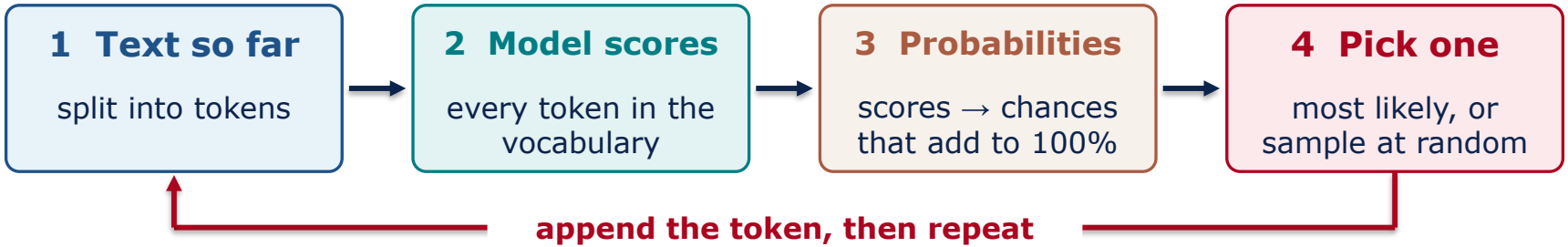
A language model gives a probability to every possible next token

- **Language model:** given the text so far, a probability for each possible next token
- It learns these probabilities from enormous amounts of text
- "formwork" and "forms" are likely; "bucket" is possible; "banana" is near zero
- Everything an LLM writes — answers, code, essays — comes from repeating this one prediction



Next-token prediction

Generating text = predict, pick, append, repeat



Step 1 The concrete was poured into the → **formwork**

Step 2 The concrete was poured into the formwork → **and**

Step 3 … poured into the formwork and → **left**

… and so on, until an end-of-text token or a length limit

Next-token prediction

Be the language model

3 min

Think · Pair · Share

- Write the **next word** for each sentence below — and how sure you are (%)
- Compare with a neighbor: where did you agree? Where did you not?
- Which sentence has **one** obvious answer, and which has **many**?

(1) Simply supported beam, uniform load q (kN/m): $M = qL^2$ divided by ___

(2) Concrete compressive strength is usually specified at ___ days

(3) The building's HVAC system failed because ___

02

Inside the model

Tokens, embeddings, and attention

Tokens

A tokenizer cuts text into pieces called tokens

Text	Qwen2.5 tokens (· = a space)	Count
“concrete”	`concrete`	1 token
“superplasticizer”	`super` `pl` `astic` `izer`	4 tokens
“cross-laminated timber”	`cross` `-l` `amin` `ated` `timber`	5 tokens
“photovoltaic”	`phot` `ovolta` `ic`	3 tokens
“29 CFR 1926.652”	`·` `2` `9` `CFR` `·` `1` `9` `2` `6` `.` `6` `5` `2`	13 tokens

Common word → 1 token · rare term → several pieces · number → 1 token per digit

Tokens

Why tokens matter to an engineer

$ Cost and limits are in tokens

Prices, speed, and context limits count tokens. OSHA Subpart P: 11,089 words → 14,891 Qwen tokens.

Numbers are split into digits

4000 psi → 4 | 0 | 0 | 0 | psi. The model never sees “4000” as one quantity — one reason arithmetic slips.

Aa Jargon is built from pieces

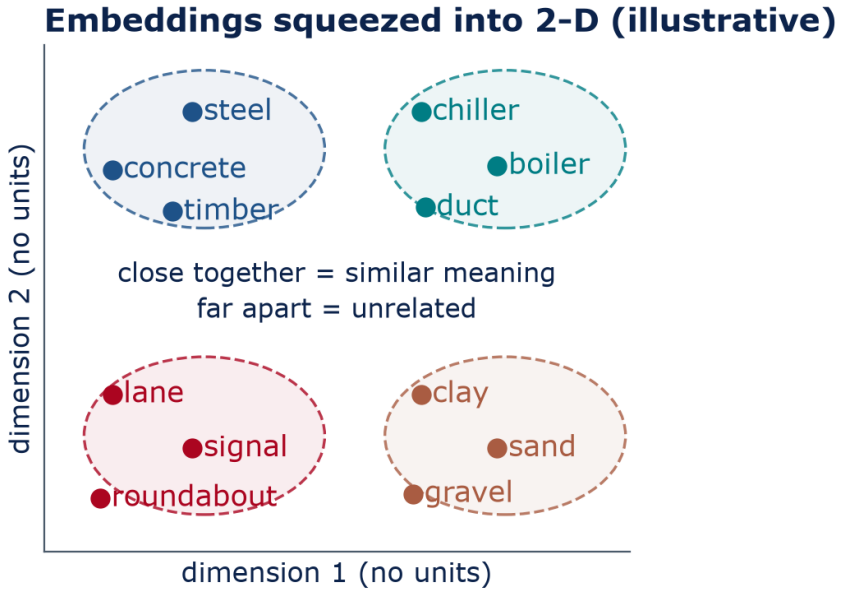
Rare terms are assembled from fragments; the model saw them less often in training, so it knows them less well.

≈ Each model has its own tokenizer

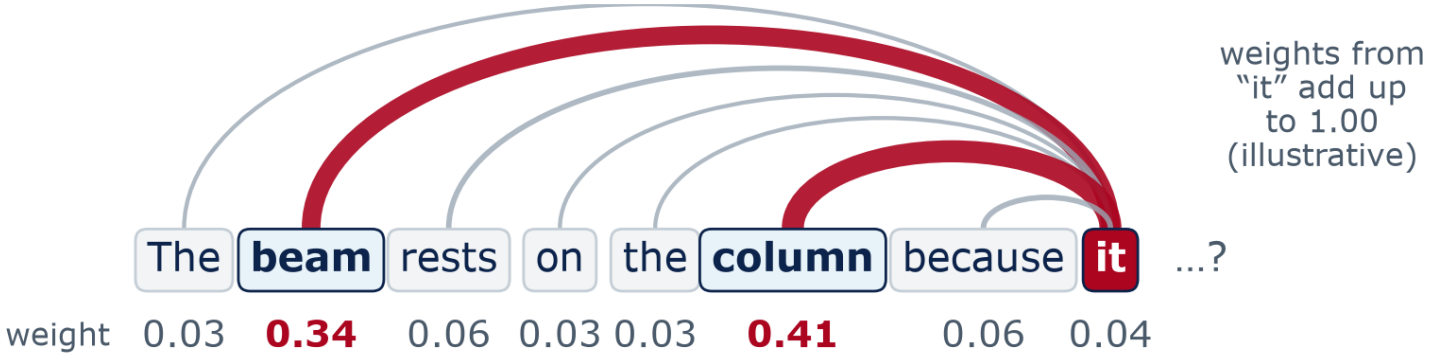
The same text gives different counts in different models. Rule of thumb for English: 1 token ≈ ¾ of a word.

Embeddings turn each token into a list of numbers

- **Embedding:** a vector (a list of numbers) that represents a token
- Learned in training: tokens used in similar contexts get **nearby vectors**
- Inside an LLM every token gets one: 896 numbers in Qwen2.5-0.5B (the plot squeezes this to 2-D)
- For search, a sentence model like MiniLM (Lab 08) gives one 384-number vector per passage



Attention: each token looks back at the others to decide what matters



1 Score

"it" scores how relevant each earlier token is. The scoring rule is learned in training — nobody programs "pronouns look at nouns"

2 Weigh

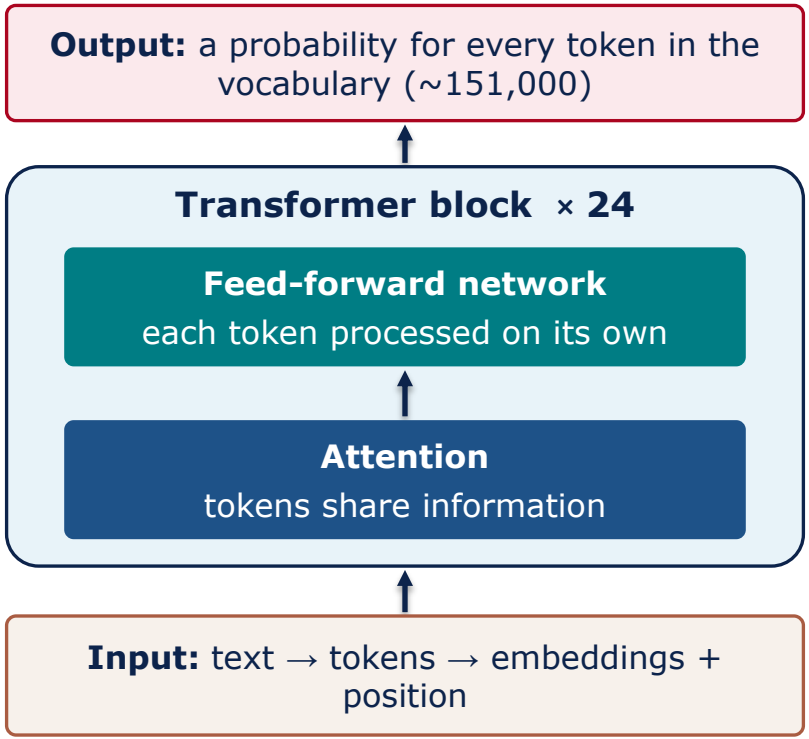
Softmax turns scores into weights that add up to 1 (the many-class cousin of Week 4's sigmoid)

3 Blend

"it" takes a weighted mix of their information — now it carries "beam" and "column"

Inside the model

A transformer stacks the same block many times



- **Attention** mixes information **between** tokens
- **Feed-forward** (an MLP, like Week 5) transforms each token **on its own**
- Shortcut connections and normalization keep a deep stack trainable
- Roughly: early blocks track local patterns; later blocks, more abstract meaning
- Qwen2.5-0.5B: 24 blocks, 14 attention heads, 896 numbers per token

03

How an LLM is trained

Same recipe as Week 3 — at enormous scale

Training

Three training stages turn a text predictor into an assistant

Pretraining

- **Data:** trillions of tokens of web text, books, code
- **Self-supervised:** the next word is the label
- **Result:** a base model that continues text

Instruction tuning

- **Data:** prompts paired with good answers written by people
- **Goal:** imitate those answers
- **Result:** follows instructions, chats

Preference tuning (RLHF)

- **Data:** people rank pairs of answers
- **Goal:** produce answers people prefer
- **Result:** more helpful, polite, cautious

Each stage uses the same engine: a cost function + gradient descent (Weeks 3–4)

Training

Tuning teaches the model to answer — and to please

Instruction tuning

- People write example prompts with ideal answers
- e.g., “Summarize this RFI (request for information) in 3 bullets”
- The model learns the chat format and to follow instructions
- Same cost as pretraining, on a much smaller, curated set

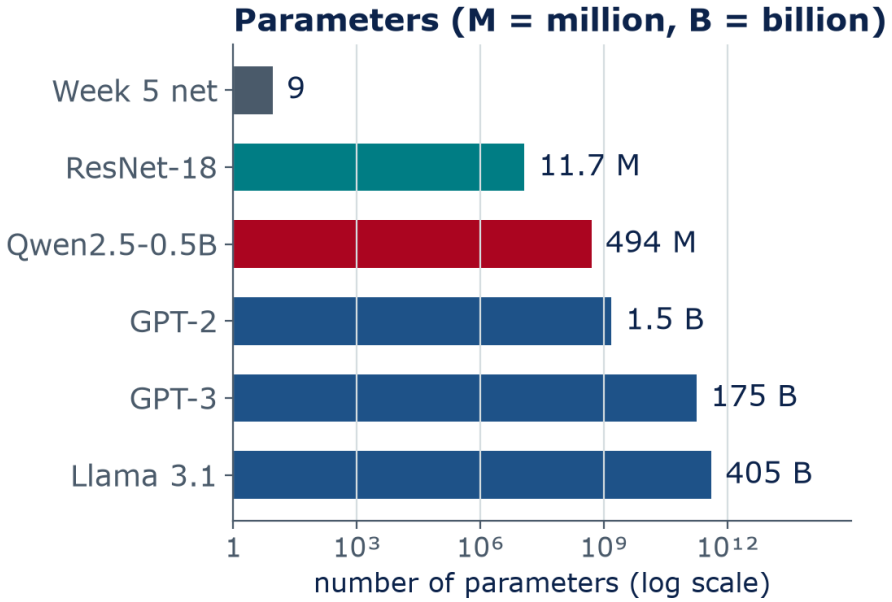
Preference tuning (RLHF)

- The model writes two answers; a person picks the better one
- A **reward model** learns to predict those choices
- The LLM is tuned to earn higher reward
- Side effect: it can learn to sound confident and agreeable, not just correct

Scale

Bigger models trained on more text are usually more capable

- Three things grow together: **parameters**, **training tokens**, **compute**
- Qwen2.5-0.5B: 494 million parameters, pretrained on up to 18 trillion tokens
- Some skills seem to appear only at large scale; researchers debate how sudden this is
- Many commercial models no longer publish their size (as of 2026)



04

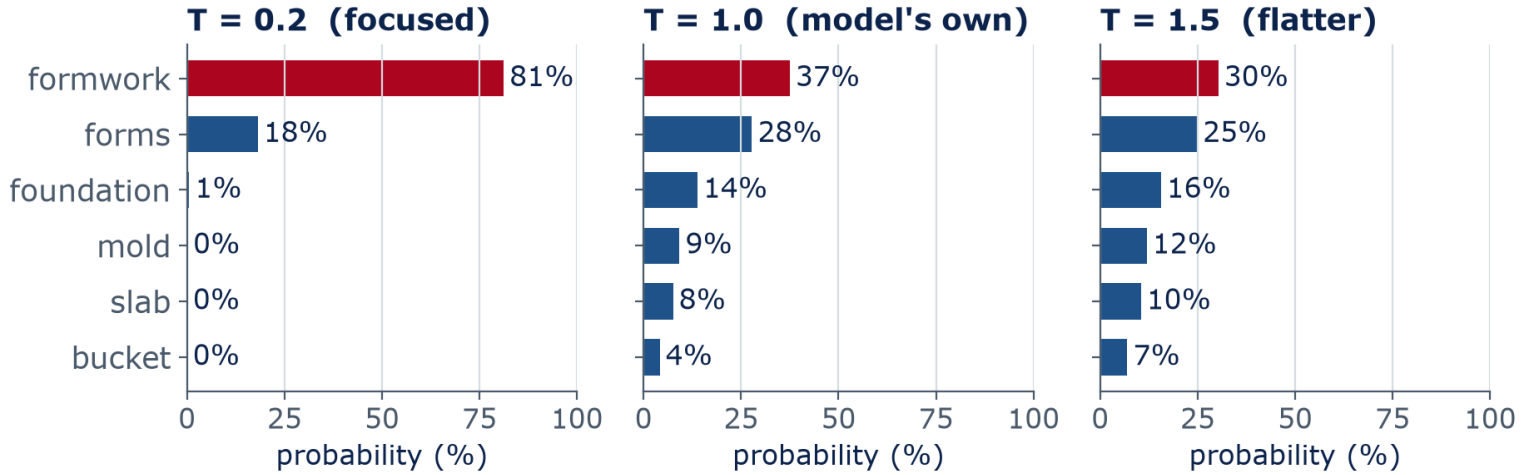
Generating text

Temperature, the context window — and why LLMs hallucinate

Generation

Temperature: low is focused, high takes chances

Next word after "The concrete was poured into the ____" at three temperatures (illustrative scores)



Temperature T divides every score before softmax: T < 1 sharpens (repeatable), T > 1 flattens (varied). T → 0 always picks the top token — our labs' `do_sample=False`.

Concept check: pick the temperature

You use an LLM to pull the specified concrete strength from 200 spec sheets into a table. Which setting makes the most sense?

A Temperature near 0, for repeatable output

B Temperature 1.5, so it considers more options

C Temperature doesn't affect the output

D As high as possible, to improve accuracy

Concept check: pick the temperature — answer

You use an LLM to pull the specified concrete strength from 200 spec sheets into a table. Which setting makes the most sense?

A Temperature near 0, for repeatable output

B Temperature 1.5, so it considers more options

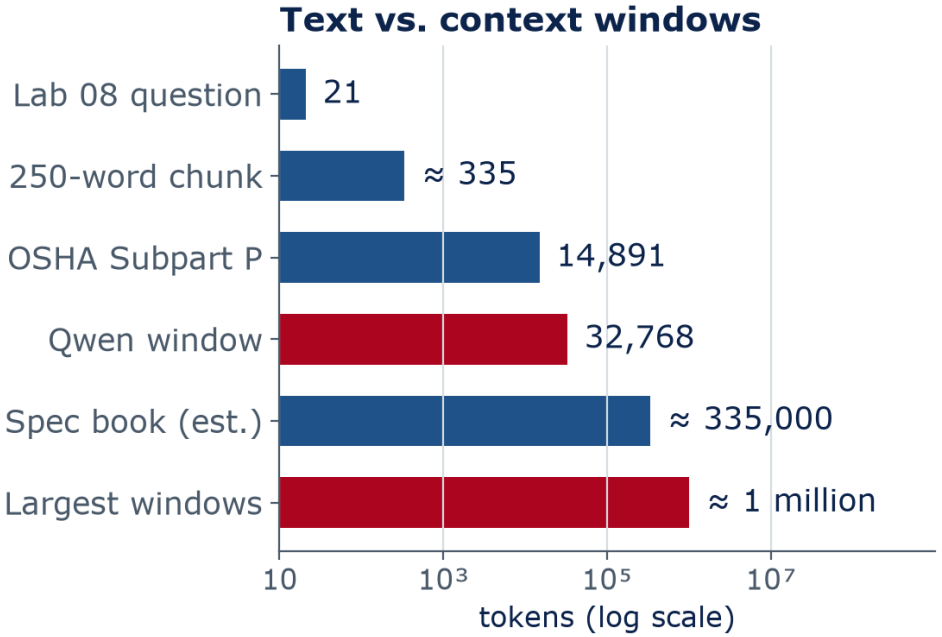
C Temperature doesn't affect the output

D As high as possible, to improve accuracy

Low T gives consistent, repeatable output — not guaranteed-correct output. Still spot-check values against the sheets.

The context window is everything the model can see at once

- **Context window:** the most tokens the model can consider — prompt, pasted documents, and its answer
- Text outside the window is simply not seen; chat apps drop or summarize old turns
- Long inputs cost more, run slower, and details in the middle can be missed
- No memory between chats except what is put back into the window



LLMs generate plausible text — nothing checks it against facts

Hallucination

When a model produces fluent, confident output that is false or not supported by any source.

- **Why:** training rewards likely text, not true text; nothing looks facts up
- **Gaps:** rare specifics, like a clause number, get filled with a plausible pattern
- **Pressure:** tests often reward a confident guess over “I don't know”

Hallucination

What hallucinations look like in engineering work

§ Invented code citations

A real-looking section or table number that doesn't exist — or a real one quoted wrongly

u Unit slips

“4000 psi ≈ 4000 MPa”; mixing kN·m and kip·ft. (4000 psi ≈ 27.6 MPa)

↺ Backwards physics

Lab 01: the small model said more water per unit cement makes stronger concrete — the opposite is true

÷ Arithmetic drift

Multi-step numbers go wrong quietly. Check $qL^2/8 = 54$ kN·m with a calculator, not the chat

? Fake references

Plausible paper titles, authors, and DOIs that were never published

! Out-of-date facts

Training stops at a cutoff date; it may quote a superseded code edition

Hallucination

Spot the errors in this AI answer

4 min

Pair work

- Read the answer below (a constructed example, written to look like a chat reply)
- With a partner, find **three** errors
- For each error: how would you **verify** it without asking the AI?

"For a simply supported beam spanning 6 m with a uniform load of 12 kN/m, the maximum bending moment is M = qL²/4 = 108 kN·m (q = load), which occurs at the supports. For the slab, f'c = 4,000 psi, or about 400 MPa."

Strengths and limits

Use LLMs for drafts and ideas; verify anything that goes into a decision

Usually good at

- Drafting and rewriting: emails, reports, meeting notes
- Summarizing long documents you provide
- Explaining concepts at different levels
- Writing and explaining short code
- Brainstorming options and checklists

vs

Often weak at

- Exact arithmetic and unit conversions
- Exact code clauses, table values, citations
- Recent facts and current code editions
- Knowing what it doesn't know
- Taking responsibility — that stays with you

Rule of thumb: the more a wrong answer would cost, the more you must check it yourself

Midterm exam

Lab day: midterm exam on Weeks 1–6

MIDTERM

75

minutes · 20%

When: lab day

Study: GitHub `exams/` folder

What it covers

- Weeks 1–6: AI map, regression and cost, gradient descent, neural nets, CNNs, sequences
- Not today's LLM material

Bring · rules

- Calculator and pencil
- Closed book · AI-Free
- No phones or laptops
- Formula sheet provided

Format

- Multiple choice + short answer
- Short calculations: a cost value, one gradient-descent step, a tiny forward pass, a parameter count
- Reading a learning curve or a confusion matrix

Wrap-up

Key takeaways

1. An LLM is a **next-token predictor**: predict, pick, append, repeat
2. Text → **tokens** → **embeddings**; similar meanings get nearby vectors
3. **Attention** lets tokens share information; a transformer stacks attention + feed-forward blocks
4. Trained with the Week 3 recipe — a cost + gradient descent — at huge scale, then tuned to follow instructions
5. Output is plausible, not verified; temperature sets randomness, not truth — **check numbers and citations**

Learn more

Readings and resources

- **3Blue1Brown — Large Language Models explained briefly** — REQUIRED (8 min) — the whole lecture in one short video
- **3Blue1Brown — Transformers (Deep Learning Ch. 5) and Attention (Ch. 6)** — optional, ≈ 27 min each; visual intuition
- **Jay Alammar — The Illustrated Transformer** — optional, ≈ 30 min read; attention step by step
- **Google ML Crash Course — Intro to Large Language Models** — optional; tokens, transformers, and limits
- **Transformer Explainer (interactive)** — optional; type a prompt, watch probabilities and temperature

Backup slides

For questions after class — not presented live

Backup

Pretraining minimizes a cost: surprise at the true next token

$$J(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log\ p_\theta(t_i \mid t_1, \ldots, t_{i-1})$$

J(θ)	cost (cross-entropy) — lower is better, like J(w, b) in Week 3
θ	all the parameters (weights) of the network
t_i	the token that actually came next in the training text
p	the probability the model gave to that true token
N	number of tokens averaged over

Backup

Quick calculation: GPT-3 on your laptop?

4 min

Self-check

- Rule of thumb: training compute ≈ **6 × N × D** operations (N = parameters, D = training tokens)
- GPT-3: N = 175 billion, D = 300 billion tokens. Compute the operations
- A laptop does about 10^{12} operations per second (generous). How many **years**?
- Use 1 year ≈ 3.15×10^{7} s

Work in three steps: **operations → seconds → years**

Answer: ≈ 3×10^{23} operations → ≈ 3×10^{11} s → ≈ 10,000 years on one laptop